

# Interview Questions for Java

## 1. Core Java:

### Theory Questions:

1. What is the difference between JDK, JRE, and JVM?
2. List the features of Java Programming language.
3. What are the various access specifiers in Java?
4. What is the purpose of static methods and variables?
5. What is the constructor? And Types of constructor Explain.
6. Does constructor return any value?
7. Can you make a constructor final?
8. What are the differences between the constructors and methods?
9. What is the static method?
10. Can we override the static methods?
11. What is Array? And Explain Types of Array.
12. What is String? And Explain memory allocation of string data.
13. Why String is immutable in JAVA?
14. How can we make String as Mutable?
15. What is OOPS?
16. Is there any procedure to execute OOPS concepts in JAVA?
17. What is Class?
18. What is Object?
19. What is Inheritance? Explain types of inheritance.
20. Which inheritance is not supported by JAVA? And why?
21. What is Polymorphism and types of polymorphism Explain.
22. Explain use of super keyword in java.
23. Explain Abstraction and how data hide from user?
24. Can we make abstract class as final? Why?
25. What is Encapsulation? And how to use data of an object instead of hashcode?
26. Explain final keyword.
27. What is Interface? How interface solve issue of multiple inheritance in JAVA?
28. Difference between Abstract class and Interface.
29. What is Exception? Types of exception by Oracle.
30. Explain keyword using which we can handle exception by different ways.
31. Explain user-define exception.
32. What is serialization and de-serialization.
33. What is Thread? How many ways are there using which we can achieve thread?
34. Which way is better to achieve thread and why?
35. What is Collection-Framework?
36. Difference between array and ArrayList.
37. Difference between List and Set.
38. Difference between ArrayList and Vector.
39. Difference between HashSet and TreeSet.
40. Explain Map and Entry Interface.
41. Difference between HashMap and HashTable.
42. What is use of generics?

### Practical Questions:

1. Write a program to demonstrate method overloading and method overriding.
2. Write a Java program to implement multithreading using `Runnable` interface.
3. Implement a Java program that uses exception handling to manage a divide-by-zero error.
4. Write a Java program to read data from a file using `BufferedReader`.
5. Write a program to create a custom exception and demonstrate its usage.

---

## 2. Advance Java:

### Theory Questions:

1. What is JDBC and what are the different types of JDBC drivers?
2. Explain the concept of servlets. How do servlets handle HTTP requests?
3. What is the difference between `GET` and `POST` methods in HTTP?
4. How does the session management work in Java EE?
5. Explain the role of `JSP` in Java web applications.
6. What is the Model-View-Controller (MVC) architecture and how is it implemented in Java?
7. Explain the concept of connection pooling in Java.
8. What are the types of elements with Java Server Pages (JSP)?
9. Explain the Life Cycle of a servlet.
10. What are some of the advantages of using JSP?
11. How does JSP work?
12. What is JSTL?
13. What are JSTL Core tags used for?
14. What are Implicit JSP Objects?

### Practical Questions:

1. Write a Java program to connect to a MySQL database using JDBC and retrieve data.
2. Create a simple Java Servlet that handles `GET` and `POST` requests.
3. Develop a basic login system using JSP and Servlets.
4. Create a CRUD application using JDBC for interacting with a MySQL database.

---

## 3. Spring Framework:

### Theory Questions:

1. What is Spring Framework? What are its main features?
2. Explain the concept of Dependency Injection (DI) in Spring.
3. What is Spring Bean? Explain its lifecycle.
4. What are the different types of autowiring in Spring?
5. What is Spring AOP (Aspect-Oriented Programming)? Give a use case for AOP.
6. How does Spring handle transaction management?
7. Explain the difference between `@Component`, `@Service`, `@Repository`, and `@Controller` annotations.
8. What is the role of `ApplicationContext` in Spring?
9. What are the bean scopes available in Spring?
10. What is autowiring and name the different modes of it?
11. What are the limitations of autowiring?

### Practical Questions:

1. Create a Spring application that demonstrates Dependency Injection using `@Autowired`.
2. Implement a Spring-based application with CRUD operations using `JdbcTemplate`.
3. Demonstrate how to configure transaction management in Spring.
4. Create a simple Spring AOP example using the `@Aspect` annotation.

---

#### 4. Hibernate Framework:

##### Theory Questions:

1. What is Hibernate? How does it simplify database operations?
2. What are the advantages of Hibernate over JDBC?
3. Explain Hibernate Session and its lifecycle.
4. What is ORM in Hibernate?
5. What is a SessionFactory?
6. What is a Session in Hibernate?
7. What is HQL?
8. What is hibernate caching?
9. Can we declare the Entity class final?
10. What are the states of a persistent entity?
11. What is the difference between `get()` and `load()` in Hibernate?
12. What are the different fetching strategies in Hibernate?
13. How does Hibernate caching work? Explain first-level and second-level caching.
14. What is the significance of the `@Entity` and `@Table` annotations?
15. How do you manage relationships (One-to-One, One-to-Many) in Hibernate?

##### Practical Questions:

1. Create a simple Hibernate application that performs CRUD operations on a MySQL database.
2. Write a program to demonstrate One-to-Many mapping in Hibernate.
3. Implement Hibernate with second-level caching using `EHCache`.
4. Configure and use Hibernate Query Language (HQL) to retrieve data.

---

#### 5. Spring Boot:

##### Theory Questions:

1. What is Spring Boot and how is it different from Spring Framework?
2. What are the features of Spring Boot?
3. Differentiate between Spring and Spring Boot.
4. What does `@SpringBootApplication` annotation do internally?
5. Can we change the default port of the embedded Tomcat server in Spring boot?
6. What is `HibernateTemplate` class?
7. Can we create a non-web application in Spring Boot?
8. Describe the flow of HTTPS requests through the Spring Boot application.
9. Explain `@RestController` annotation in Spring Boot.
10. What is the purpose of using `@ComponentScan` in the class files?
11. What are the main advantages of using Spring Boot for application development?
12. How does Spring Boot simplify configuration compared to traditional Spring applications?
13. What is `application.properties` or `application.yml` in Spring Boot?
14. Explain the concept of Spring Boot Starter dependencies.
15. How does Spring Boot handle logging?
16. What is the role of Spring Boot Actuator?

### Practical Questions:

1. Create a simple Spring Boot application and expose a REST API.
2. Implement a CRUD service in Spring Boot using JPA for database operations.
3. Demonstrate how to use custom properties in `application.properties` and access them in a Spring Boot application.
4. Set up and run a Spring Boot application with Spring Boot Actuator.

---

## 6. REST API:

### Theory Questions:

1. What is REST and what are its architectural principles?
2. Can you tell the disadvantages of RESTful web services?
3. What are HTTP Status codes?
4. What are Idempotent methods? How is it relevant in RESTful web services domain?
5. How does HTTP Basic Authentication work?
6. Explain the HTTP methods used in REST (GET, POST, PUT, DELETE).
7. What is the difference between REST and SOAP?
8. How does REST handle error messages and status codes?
9. What is idempotency and how does it apply to HTTP methods?
10. What is the purpose of `@RestController` annotation in Spring Boot?

### Practical Questions:

1. Create a REST API in Spring Boot to perform basic CRUD operations.
2. Implement request validation in a Spring Boot REST API.
3. Create a RESTful service that interacts with a MySQL database using Spring Data JPA.
4. Write a Spring Boot REST API with pagination and sorting.

---

## 7. Microservices:

### Theory Questions:

1. What are microservices and how do they differ from monolithic architectures?
2. What issues are generally solved by spring clouds?
3. Write main components of Microservices.
4. What are the main benefits of using a microservice architecture?
5. Explain the role of API Gateway in a microservice architecture.
6. What is service discovery in microservices and how is it implemented?
7. How do microservices handle inter-service communication?
8. What is circuit breaker pattern and how is it implemented in microservices?

### Practical Questions:

1. Create a basic microservice in Spring Boot and expose an API.
2. Implement inter-service communication between two microservices using REST.
3. Set up a Eureka server for service discovery in a microservice-based application.
4. Implement circuit breaker using `Hystrix` in a Spring Boot microservice.